

# NEXUS-GUARDIAN Frontend Development Prompt

## Project Overview

Create a modern, professional web interface for **NEXUS-GUARDIAN v4.0** - an ultra-advanced AI/ML fraud detection and honeypot engagement system with 12 layers of neural networks. This system detects scam messages, engages fraudsters autonomously, and extracts intelligence using a mathematically impenetrable dual-LLM architecture.

---

## System Architecture Understanding

### Backend Capabilities

The Python backend provides:

#### 1. 12-Layer Neural Network Stack:

- L0: Adversarial Input Sanitization
- L1: Temporal Graph Neural Network (TGNN)
- L2: Transformer with Multi-Head Attention
- L3: Bidirectional LSTM
- L4: Hybrid CNN-RNN
- L5: Graph Attention Network (GAT)
- L6: Variational Autoencoder (VAE)
- L7: Deep Q-Network (DQN) RL Agent
- L8: Gradient Boosting
- L9: Bayesian Neural Network
- L10: Meta-Learning Aggregator
- L11: Dual-LLM Air-Gapped Cognitive Core

#### 2. Dual-LLM Architecture (Unhackable):

- **Supervisor Agent:** Strategic decision-making (unreachable from external input)
- **Persona Agent:** Scammer engagement (quarantined, isolated from internals)

#### 3. Intelligence Extraction:

- Phone numbers, bank accounts, UPI IDs
- URLs, email addresses, IP addresses
- Fraud patterns and tactics

Key API Endpoints (Expected)

python

POST /api/analyze

```
{
  "session_id": "session-xxx",
  "message": "user message text",
  "conversation_history": [...],
  "metadata": {"timestamp": 1234567890000}
}
```

Response:

```
{
  "session_id": "session-xxx",
  "total_fraud_score": 0.87,
  "confidence": 0.94,
  "uncertainty": 0.06,
  "risk_level": "HIGH",
  "model_scores": {
    "tgnn": 0.85,
    "transformer": 0.89,
    ...
  },
  "recommended_action": "SOFT_BLOCK",
  "intelligence_extracted": {
    "phone_numbers": ["+91-9876543210"],
    "upi_ids": ["scammer@paytm"],
    ...
  },
  "engagement_strategy": {...}
}
```

POST /api/engage-honeypot

```
{
  "session_id": "session-xxx",
  "conversation_history": [...]
}
```

Response:

```
{
  "reply": "AI-generated response",
  "typing_latency": 2.5,
  "should_continue": true,
  "intelligence_value": 0.75,
}
```

```
"total_messages": 5
```

```
}
```

## Frontend Requirements

### Core Pages/Views

#### 1. Dashboard (Main View)

**Purpose:** Real-time monitoring and system overview

#### Components:

- **Hero Statistics Panel** (Top Cards):
  - Total Messages Analyzed (with trend)
  - Fraud Detection Rate (%)
  - Active Honeypot Sessions
  - Intelligence Extracted
- **Real-Time Activity Feed:**
  - Live stream of analyzed messages
  - Color-coded by risk level (GREEN/YELLOW/ORANGE/RED)
  - Show: timestamp, message snippet, fraud score, risk level
- **Risk Distribution Chart:**
  - Pie/donut chart showing: SAFE, LOW, MEDIUM, HIGH, CRITICAL
  - Animated transitions
- **Neural Network Layer Visualization:**
  - 12 horizontal bars/gauges showing each layer's contribution
  - Animated score updates
  - Tooltips explaining each layer
- **Active Honeypot Conversations:**
  - List of ongoing engagements
  - Time elapsed, messages exchanged, intelligence extracted
  - Click to view detailed conversation

#### Design Style:



- Dark theme with neon accents (cyber security aesthetic)
- Glassmorphism cards with subtle gradients
- Smooth animations and transitions
- Monospace fonts for technical data
- Real-time data updates without page refresh

## 2. Message Analyzer

**Purpose:** Interactive fraud detection testing interface

**Layout:**

Message Input Panel

Enter message to analyze...

[Analyze Message]

[Clear]

Analysis Results

Fraud Score

87.3%

Risk Level

HIGH

Neural Network Scores:

TGNN

85%

Transformer

89%

LSTM

78%

...

Extracted Intelligence:

Phone: +91-9876543210

UPI: scammer@paytm

URL: http://phishing-site.com

**Features:**

- Auto-expanding textarea
  - Example fraud messages (dropdown presets)
  - Real-time analysis (debounced input)
  - Detailed breakdown of why message flagged
  - Confidence intervals with uncertainty visualization
  - Copy/export results button
  - Mathematical formulas shown in tooltips
- 

**3. Honeypot Engagement Studio**

**Purpose:** Monitor and control AI-powered honeypot conversations

**Features:**

- **Conversation List** (Left Sidebar):
  - Active sessions with status indicators
  - Sort by: newest, highest fraud score, most intelligence
  - Filter by risk level
- **Chat Interface** (Center):
  - Two-column view:
    - Left: Scammer messages
    - Right: AI Agent responses
  - Timestamps and latency indicators
  - Message-by-message fraud score evolution graph
  - Typing indicators during AI response generation
- **Intelligence Panel** (Right Sidebar):
  - Extracted data in real-time
  - Categorized sections (phones, accounts, URLs, etc.)
  - Validation status indicators
  - Export intelligence button
  - Fraud pattern analysis
- **Control Panel** (Bottom):

- Manual override options
- Engagement strategy settings
- "End Session" / "Continue" buttons
- Intelligence extraction toggle

### Special Features:

- Syntax highlighting for extracted data
  - Animated graph showing conversation flow
  - Real-time updates via WebSocket/polling
  - Session recording and playback
- 

## 4. Intelligence Database

**Purpose:** Searchable repository of extracted fraud intelligence

### Components:

- **Search & Filter Bar:**
  - Full-text search
  - Filters: date range, data type, fraud score
  - Sort options
- **Data Grid/Table:**
  - Columns: Timestamp, Type, Value, Source Session, Fraud Score, Validation Status
  - Pagination
  - Export to CSV/JSON
- **Statistics Dashboard:**
  - Most common fraud patterns
  - Geographic distribution (if phone numbers)
  - Temporal trends (fraud activity over time)
  - Network graph visualization (connected entities)
- **Detail Modal:**
  - Click any entry to see full context
  - Original conversation
  - All related extracted data

- Fraud analysis breakdown
- 

## 5. System Configuration

**Purpose:** Adjust detection thresholds and system parameters

**Sections:**

### 1. Detection Thresholds:

- Sliders for: CRITICAL (0.90), HIGH (0.75), MEDIUM (0.50), LOW (0.25)
- Real-time impact preview
- Reset to defaults

### 2. Neural Network Weights:

- Adjust contribution of each layer
- Visual weight distribution chart
- "Meta-Learning Auto-Optimize" button

### 3. Honeypot Configuration:

- Max conversation turns (default: 20)
- Engagement strategy (aggressive/passive/adaptive)
- Auto-terminate conditions
- Typing latency settings (human-like delays)

### 4. API Settings:

- Rate limiting
- Authentication tokens
- Callback URLs (for GUVI integration)

### 5. Model Performance:

- Live metrics: Precision, Recall, F1 Score
  - Adversarial robustness indicator
  - Lipschitz constant estimate
  - Model retraining controls
-

Color Palette

CSS

```
/* Dark Theme - Cyber Security Aesthetic */
--background-primary: #0a0e1a;
--background-secondary: #151b2e;
--background-tertiary: #1e2840;

--accent-safe: #10b981;    /* Green - Safe */
--accent-low: #3b82f6;     /* Blue - Low Risk */
--accent-medium: #f59e0b;  /* Amber - Medium */
--accent-high: #ef4444;    /* Red - High */
--accent-critical: #dc2626; /* Dark Red - Critical */

--accent-neon-blue: #00d9ff;
--accent-neon-purple: #b175ff;
--accent-neon-pink: #ff0080;

--text-primary: #f8fafc;
--text-secondary: #cbd5e1;
--text-muted: #64748b;

--border-subtle: rgba(255, 255, 255, 0.1);
--glass-background: rgba(255, 255, 255, 0.05);
--glass-border: rgba(255, 255, 255, 0.1);
```

Typography

CSS

```
/* Font Stack */
--font-primary: 'Inter', -apple-system, BlinkMacSystemFont, sans-serif;
--font-mono: 'Fira Code', 'Monaco', 'Courier New', monospace;
--font-display: 'Space Grotesk', sans-serif;
```

```
/* Sizes */
--text-xs: 0.75rem; /* 12px */
--text-sm: 0.875rem; /* 14px */
--text-base: 1rem; /* 16px */
--text-lg: 1.125rem; /* 18px */
--text-xl: 1.25rem; /* 20px */
--text-2xl: 1.5rem; /* 24px */
--text-3xl: 1.875rem; /* 30px */
--text-4xl: 2.25rem; /* 36px */
```

## Component Patterns

### Glassmorphism Card

CSS

```
.glass-card {
  background: rgba(255, 255, 255, 0.05);
  backdrop-filter: blur(10px);
  border: 1px solid rgba(255, 255, 255, 0.1);
  border-radius: 16px;
  box-shadow: 0 8px 32px rgba(0, 0, 0, 0.3);
}
```

### Neon Glow Button

CSS

```
.neon-button {
  background: linear-gradient(135deg, #00d9ff, #b175ff);
  box-shadow: 0 0 20px rgba(0, 217, 255, 0.5);
  transition: all 0.3s ease;
}

.neon-button:hover {
  box-shadow: 0 0 30px rgba(0, 217, 255, 0.8);
  transform: translateY(-2px);
}
```

## Risk Level Badge

```
jsx

// Dynamic color based on risk
const getRiskColor = (level) => {
  const colors = {
    SAFE: '#10b981',
    LOW: '#3b82f6',
    MEDIUM: '#f59e0b',
    HIGH: '#ef4444',
    CRITICAL: '#dc2626'
  };
  return colors[level];
};
```

---

## Technical Stack Recommendations

### Frontend Framework

**React** with TypeScript (preferred) or **Vue 3**

### Key Libraries

#### UI & Styling

- **Tailwind CSS** - Utility-first styling
- **Framer Motion** - Smooth animations
- **Radix UI** or **Headless UI** - Accessible components
- **Lucide React** - Icon library

### Data Visualization

- **Recharts** or **Chart.js** - Charts and graphs
- **D3.js** - Complex visualizations (network graphs)
- **React Flow** - Node-based flow diagrams (for neural network visualization)

### State Management

- **Zustand** or **Redux Toolkit** - Global state
- **React Query** or **SWR** - Server state & caching

## Real-time Updates

- **Socket.io** (WebSocket) or **Server-Sent Events (SSE)**
- Polling fallback for real-time dashboard updates

## Form & Validation

- **React Hook Form** - Form handling
- **Zod** - Schema validation

## Utilities

- **date-fns** or **Day.js** - Date formatting
  - **clsx** - Conditional classnames
  - **react-hot-toast** - Notifications
- 

## Animation & Interactions

### Key Animations

1. **Fraud Score Counter:**
  - Animated number counting from 0 to score
  - Color transition based on threshold crossed
  - Pulse effect on high scores
2. **Neural Network Layer Bars:**
  - Staggered animation (each bar animates sequentially)
  - Glow effect on active layers
  - Smooth width transitions
3. **Real-time Feed:**
  - New items slide in from top
  - Fade out items after 50 entries
  - Highlight flash on new fraud detection
4. **Loading States:**
  - Skeleton screens (not spinners)
  - Progress indicators for analysis



- Typing animation for AI responses

## 5. Page Transitions:

- Smooth fade + slide
  - Shared element transitions where applicable
- 

## Responsive Design

### Breakpoints

- Mobile: 0-640px
- Tablet: 641px-1024px
- Desktop: 1025px+

### Mobile Considerations

- Collapsible sidebar navigation
  - Stacked card layouts
  - Bottom sheet for detail views
  - Touch-friendly hit targets (min 44px)
  - Simplified charts for mobile
- 

## Security & Best Practices

### Frontend Security

- **Input Sanitization:** Escape all user inputs
- **XSS Prevention:** Use framework's built-in protections
- **CSRF Protection:** Include tokens in API requests
- **Secure Storage:** Never store API keys in frontend
- **Content Security Policy:** Implement CSP headers

### Performance Optimization

- **Code Splitting:** Lazy load routes
- **Image Optimization:** WebP format, lazy loading

- **Bundle Size:** Keep initial bundle < 200KB
- **Caching:** Aggressive caching for static assets
- **Debouncing:** For search and real-time inputs

**Accessibility (WCAG 2.1 AA)**

- Keyboard navigation for all interactions
  - ARIA labels for screen readers
  - Color contrast ratios (min 4.5:1 for text)
  - Focus indicators visible
  - Error messages programmatically associated
- 

 **Data Flow Example**

```
mermaid
graph TD
    A[User enters message] --> B[Frontend validates input]
    B --> C[POST /api/analyze]
    C --> D[Backend: 12 Neural Layers]
    D --> E[Return fraud analysis]
    E --> F[Frontend: Update UI]
    F --> G{Fraud Score > 0.75?}
    G -->|Yes| H[Show Honeypot Option]
    G -->|No| I[Display Safe Result]
    H --> J[User clicks Engage]
    J --> K[POST /api/engage-honeypot]
    K --> L[Real-time conversation updates]
    L --> M[Extract intelligence]
    M --> N[Update Intelligence DB]
```

 **Component Examples**

**1. Fraud Score Gauge Component**

```
jsx
```

```

import { motion } from 'framer-motion';

const FraudScoreGauge = ({ score, confidence }) => {
  const getRiskLevel = (score) => {
    if (score >= 0.90) return { level: 'CRITICAL', color: '#dc2626' };
    if (score >= 0.75) return { level: 'HIGH', color: '#ef4444' };
    if (score >= 0.50) return { level: 'MEDIUM', color: '#f59e0b' };
    if (score >= 0.25) return { level: 'LOW', color: '#3b82f6' };
    return { level: 'SAFE', color: '#10b981' };
  };

  const { level, color } = getRiskLevel(score);

  return (
    <div className="glass-card p-6">
      <h3 className="text-sm text-text-secondary mb-4">Fraud Detection</h3>

      { /* Circular Progress */ }
      <div className="relative w-32 h-32 mx-auto">
        <svg className="transform -rotate-90" viewBox="0 0 100 100">
          { /* Background circle */ }
          <circle
            cx="50" cy="50" r="45"
            fill="none"
            stroke="rgba(255,255,255,0.1)"
            strokeWidth="8"
          />
          { /* Progress circle */ }
          <motion.circle
            cx="50" cy="50" r="45"
            fill="none"
            stroke={color}
            strokeWidth="8"
            strokeLinecap="round"
            initial={{ pathLength: 0 }}
            animate={{ pathLength: score }}
            transition={{ duration: 1, ease: "easeOut" }}
            style={{
              pathLength: score,
              filter: `drop-shadow(0 0 10px ${color})`
            }}
          />
        </svg>
      </div>
    </div>
  );
};

```

```

    {/ * Center text */}
    <div className="absolute inset-0 flex flex-col items-center justify-center">
      <motion.span
        className="text-3xl font-bold"
        style={{ color }}
        initial={{ opacity: 0, scale: 0.5 }}
        animate={{ opacity: 1, scale: 1 }}
        transition={{ delay: 0.5 }}
      >
        {(score * 100).toFixed(1)}%
      </motion.span>
      <span className="text-xs text-text-muted">confidence: {(confidence * 100).toFixed(0)}%</span>
    </div>
  </div>

```

```

    {/ * Risk Level Badge */}
    <div className="mt-4 text-center">
      <span
        className="inline-block px-4 py-2 rounded-full text-sm font-semibold"
        style={{
          backgroundColor: `${color}20`,
          color: color,
          border: `1px solid ${color}`
        }}
      >
        {level}
      </span>
    </div>
  </div>
);
};

```

## 2. Neural Network Layers Visualization

jsx

```

const NeuralLayerScores = ({ scores }) => {
  const layers = [
    { name: 'TGNN', key: 'tgnn', description: 'Temporal Graph Analysis' },
    { name: 'Transformer', key: 'transformer', description: 'Attention Mechanism' },
    { name: 'Bi-LSTM', key: 'lstm', description: 'Sequence Analysis' },
    { name: 'CNN-RNN', key: 'cnn_rnn', description: 'Pattern Recognition' },
    { name: 'GAT', key: 'gat', description: 'Graph Attention' },
    { name: 'VAE', key: 'vae', description: 'Anomaly Detection' },
    { name: 'DQN', key: 'dqn', description: 'RL Agent' },
    { name: 'Gradient Boost', key: 'gb', description: 'Ensemble Learning' },
    { name: 'Bayesian NN', key: 'bayesian', description: 'Uncertainty Quantification' },
  ];

  return (
    <div className="glass-card p-6">
      <h3 className="text-lg font-semibold mb-4">Neural Network Analysis</h3>

      <div className="space-y-3">
        {layers.map((layer, index) => (
          <motion.div
            key={layer.key}
            initial={{ opacity: 0, x: -20 }}
            animate={{ opacity: 1, x: 0 }}
            transition={{ delay: index * 0.05 }}
          >
            <div className="flex items-center justify-between mb-1">
              <span className="text-sm font-medium text-text-primary">
                {layer.name}
              </span>
              <span className="text-xs text-text-muted">
                {((scores[layer.key] || 0) * 100).toFixed(1)}%
              </span>
            </div>

            { /* Progress Bar */ }
            <div className="h-2 bg-background-tertiary rounded-full overflow-hidden">
              <motion.div
                className="h-full bg-gradient-to-r from-accent-neon-blue to-accent-neon-purple"
                initial={{ width: 0 }}
                animate={{ width: `${(scores[layer.key] || 0) * 100}%` }}
                transition={{ duration: 0.8, delay: index * 0.05 }}
                style={{
                  boxShadow: '0 0 10px rgba(0, 217, 255, 0.5)'
                }}
              />
            </div>
          </motion.div>
        ))}
      </div>
    </div>
  );
}

```

```
    }}  
  />  
</div>  
  
    <p className="text-xs text-text-muted mt-1">{layer.description}</p>  
  </motion.div>  
  )}  
</div>  
</div>  
);  
};
```

### 3. Real-time Activity Feed

```
jsx
```

```

import { useQuery } from 'react-query';
import { motion, AnimatePresence } from 'framer-motion';

const ActivityFeed = () => {
  const { data: activities } = useQuery(
    'activity-feed',
    fetchActivities,
    { refetchInterval: 2000 } // Poll every 2 seconds
  );

  return (
    <div className="glass-card p-6 h-96 overflow-hidden">
      <h3 className="text-lg font-semibold mb-4">Real-time Activity</h3>

      <div className="space-y-2 overflow-y-auto h-full pr-2">
        <AnimatePresence>
          {activities?.map((activity) => (
            <motion.div
              key={activity.id}
              initial={{ opacity: 0, y: -20, scale: 0.95 }}
              animate={{ opacity: 1, y: 0, scale: 1 }}
              exit={{ opacity: 0, x: -100 }}
              className={`
                p-3 rounded-lg border-l-4
                ${getRiskBorderColor(activity.risk_level)}
                bg-background-tertiary
              `}
            >
              <div className="flex items-start justify-between">
                <div className="flex-1">
                  <p className="text-sm text-text-primary font-mono truncate">
                    {activity.message}
                  </p>
                  <div className="flex items-center gap-2 mt-1">
                    <span className="text-xs text-text-muted">
                      {formatTimestamp(activity.timestamp)}
                    </span>
                    <span className="text-xs text-text-muted">•</span>
                    <span className="text-xs font-semibold" style={{
                      color: getRiskColor(activity.risk_level)
                    }}>
                      {(activity.fraud_score * 100).toFixed(0)}% fraud
                    </span>
                  </div>
                </div>
              </div>
            </motion.div>
          ))}
        </AnimatePresence>
      </div>
    </div>
  );
}

```

```

        </div>
      </div>

      <button className="ml-2 text-text-muted hover:text-text-primary">
        <ChevronRight size={16} />
      </button>
    </div>
  </motion.div>
  )))
</AnimatePresence>
</div>
</div>
);
};

```

## Implementation Checklist

### Phase 1: Core Setup (Week 1)

- ☐ Initialize React/Vue project with TypeScript
- ☐ Set up Tailwind CSS with custom theme
- ☐ Configure routing (React Router / Vue Router)
- ☐ Set up state management (Zustand/Redux)
- ☐ Create base layout components (Header, Sidebar, Main)
- ☐ Implement dark theme system
- ☐ Set up API client with interceptors

### Phase 2: Dashboard (Week 2)

- ☐ Build statistics cards with real-time updates
- ☐ Implement activity feed with WebSocket/polling
- ☐ Create risk distribution chart
- ☐ Build neural network layer visualization
- ☐ Add active honeypot sessions list
- ☐ Implement responsive layout

### Phase 3: Message Analyzer (Week 3)

- ☐ Build message input component
- ☐ Create analysis results display
- ☐ Implement fraud score gauge



- ☐ Add neural network breakdown
- ☐ Build intelligence extraction display
- ☐ Add example message presets

#### **Phase 4: Honeypot Studio (Week 4)**

- ☐ Build conversation list sidebar
- ☐ Create chat interface (two-column)
- ☐ Implement intelligence panel
- ☐ Add control panel with actions
- ☐ Set up real-time message updates
- ☐ Add session management

#### **Phase 5: Intelligence DB (Week 5)**

- ☐ Build search and filter system
- ☐ Create data grid with pagination
- ☐ Implement export functionality
- ☐ Add statistics dashboard
- ☐ Create detail modal
- ☐ Build network graph visualization

#### **Phase 6: Configuration (Week 6)**

- ☐ Build threshold slider controls
- ☐ Create neural weight adjustment
- ☐ Implement honeypot settings
- ☐ Add API configuration
- ☐ Build performance metrics display
- ☐ Add save/reset functionality

#### **Phase 7: Polish & Testing (Week 7)**

- ☐ Add loading skeletons
  - ☐ Implement error boundaries
  - ☐ Add toast notifications
  - ☐ Optimize performance (code splitting)
  - ☐ Test accessibility
  - ☐ Cross-browser testing
  - ☐ Mobile responsiveness refinement
-

## Success Criteria

### Functionality

- ☒ All CRUD operations working with backend
- ☒ Real-time updates (< 3 second latency)
- ☒ All visualizations rendering correctly
- ☒ Honeypot engagement flow complete
- ☒ Intelligence extraction and display

### Performance

- ☒ Initial load < 3 seconds
- ☒ Time to Interactive < 5 seconds
- ☒ Smooth 60fps animations
- ☒ No memory leaks in long sessions

### UX/Design

- ☒ Intuitive navigation
- ☒ Clear visual hierarchy
- ☒ Consistent design system
- ☒ Professional cyber-security aesthetic
- ☒ Responsive on all devices

### Code Quality

- ☒ TypeScript strict mode
- ☒ Component reusability > 70%
- ☒ Test coverage > 80%
- ☒ No critical accessibility issues
- ☒ Clean, documented code

---

## Advanced Features (Optional Enhancements)

1. **Dark/Light Theme Toggle** - User preference
2. **Custom Dashboard Builder** - Drag-and-drop widgets

3. **Advanced Search** - Natural language queries
  4. **Webhook Testing** - Test GUVI API callbacks
  5. **Model Comparison** - A/B test different configurations
  6. **Export Reports** - PDF/CSV fraud analysis reports
  7. **Team Collaboration** - Multi-user support
  8. **Audit Logs** - Track all system actions
  9. **Notification Center** - Critical alerts
  10. **API Playground** - Test endpoints interactively
- 

## Integration Notes

### Backend API Expectations

The frontend expects the Python backend to expose REST APIs at:

- Base URL: `http://localhost:8000/api` (development)
- Authentication: Bearer token in headers
- Response format: JSON

### WebSocket for Real-time

```
javascript

// Example WebSocket connection
const ws = new WebSocket('ws://localhost:8000/ws');

ws.onmessage = (event) => {
  const data = JSON.parse(event.data);

  if (data.type === 'new_analysis') {
    // Update activity feed
  } else if (data.type === 'honeypot_message') {
    // Update conversation
  } else if (data.type === 'intelligence_extracted') {
    // Update intelligence panel
  }
};
```

---

## Documentation Requirements

For each component, provide:

1. **Purpose** - What does it do?
  2. **Props/API** - Input parameters
  3. **Usage Example** - Code snippet
  4. **Accessibility Notes** - ARIA labels, keyboard nav
  5. **Performance Considerations** - Optimization tips
- 

## Mathematical Visualization

For displaying formulas (optional but impressive):

```
jsx

import 'katex/dist/katex.min.css';
import { BlockMath, InlineMath } from 'react-katex';

<BlockMath math="\Phi(x,t,G) = \int_0^t \sum_{i=0}^{11} w_i \cdot \lambda_i(x,\tau,G) d\tau" />
```

---

## Deployment

### Build Configuration

```
bash

# Production build
npm run build

# Environment variables
REACT_APP_API_URL=https://api.nexusguardian.ai
REACT_APP_WS_URL=wss://api.nexusguardian.ai/ws
REACT_APP_VERSION=4.0.0
```

### Hosting Recommendations

- **Vercel / Netlify** - Easiest for React
- **AWS Amplify** - Full-stack hosting

- **Docker** - For containerized deployment
- 

## **Support & Resources**

- **Design Inspiration:**
    - Vercel Dashboard
    - Linear App
    - Supabase Console
  - **Color Palette Tools:**
    - Colors.co
    - Realtime Colors
  - **Animation References:**
    - Framer Motion Examples
    - Aceternity UI
- 

## **Final Notes**

This prompt is designed to create a **production-ready, enterprise-grade frontend** for the NEXUS-GUARDIAN system. Focus on:

1. **User Experience** - Intuitive, fast, delightful
2. **Visual Polish** - Professional cyber-security aesthetic
3. **Performance** - Smooth, responsive, optimized
4. **Accessibility** - Inclusive for all users
5. **Maintainability** - Clean, documented, scalable code

The goal is to create an interface that matches the sophistication of the 12-layer AI/ML backend while being accessible to both technical and non-technical users.

**Good luck building the future of fraud detection!**  